

Manual do Projeto - MKT Digital Platform

Plataforma de Marketing Digital com IA

Versão: 1.0.0

Data: 09/05/2026

Stack: Next.js 16 + TypeScript + Tailwind CSS + Prisma + AWS Bedrock

1. Visão Geral

A MKT Digital Platform é uma plataforma SaaS de marketing digital que utiliza Inteligência Artificial para gerar conteúdo (texto e imagem) para redes sociais. O cliente configura sua identidade visual, tom de comunicação e a IA gera posts personalizados.

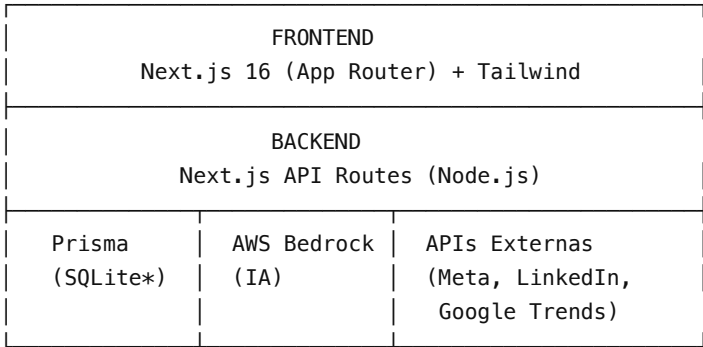
Funcionalidades implementadas:

- Autenticação (email/senha + Google OAuth)
- Onboarding com configuração da empresa (nome, setor, tom, cores, logo)
- Geração de 3 opções de texto via Claude Sonnet 4.6 (AWS Bedrock)
- Geração de 3 opções de imagem via Stable Image Core (AWS Bedrock)
- Upload de imagens de referência para contexto visual
- Busca automática de trending topics (Google Trends, G1, Google News)
- Agendamento de posts com calendário visual e recorrência
- Conexão com redes sociais (Instagram, Facebook, LinkedIn, WhatsApp)
- Publicação automática via cron job
- Dashboard de custos com rastreamento por geração (tokens, imagens, USD)

Funcionalidades pendentes (Fase 4 e 5):

- Geração de vídeo (HeyGen/Runway)
- Otimização de campanhas de tráfego pago
- Dashboard de métricas das redes sociais
- Planos de assinatura com limites mensais

2. Arquitetura



* SQLite para desenvolvimento. Migrar para PostgreSQL em produção.

Modelos de IA utilizados:

Serviço	Modelo	Região	Uso
Texto	Claude Sonnet 4.6	us-east-1	Geração de copies/legendas

Imagem	Stable Image Core v1.1	us-west-2	Geração de artes para posts
--------	------------------------	-----------	-----------------------------

3. Estrutura de Pastas

```
mkt-digital-platform/
├─ prisma/
│   ├── schema.prisma      # Schema do banco de dados
│   ├── seed.ts            # Dados de seed para demo
│   └─ dev.db              # Banco SQLite (gerado)
├─ public/
│   └─ uploads/            # Imagens de referência enviadas
├─ src/
│   ├── app/
│   │   ├── (auth)/
│   │   │   ├── login/    # Página de login
│   │   │   └─ register/  # Página de registro
│   │   ├── (dashboard)/
│   │   │   ├── costs/    # Aba de custos de IA
│   │   │   ├── create-post/ # Criação de post com IA
│   │   │   ├── dashboard/ # Dashboard principal
│   │   │   ├── onboarding/ # Configuração da empresa
│   │   │   ├── posts/     # Listagem de posts
│   │   │   ├── schedule/  # Calendário de agendamento
│   │   │   └─ social/     # Conexão de redes sociais
│   │   └─ api/
│   │       ├── auth/      # NextAuth endpoints
│   │       ├── company/   # CRUD empresa + logo
│   │       ├── costs/     # Consulta custos
│   │       ├── cron/      # Cron de publicação
│   │       ├── generate/   # Geração texto + imagem
│   │       ├── posts/     # CRUD + agendamento posts
│   │       ├── social/    # Conexão + publicação redes
│   │       ├── trends/    # Busca trending topics
│   │       └─ upload/     # Upload de imagens
│   │   ├── layout.tsx     # Layout raiz
│   │   └─ page.tsx        # Landing page
│   ├── components/
│   │   └─ auth/
│   │       └─ AuthProvider.tsx
│   └─ lib/
│       ├── auth.ts        # Configuração NextAuth
│       ├── bedrock.ts     # Integração AWS Bedrock
│       ├── image-compose.ts # Overlay de logo em imagens
│       ├── prisma.ts      # Cliente Prisma singleton
│       └─ social.ts       # Publicação nas redes sociais
├─ .env                    # Variáveis de ambiente
├─ .env.example            # Template de env vars
├─ package.json
└─ tsconfig.json
```

4. Setup do Ambiente de Desenvolvimento

Pré-requisitos:

- Node.js >= 20 (<https://nodejs.org>)
 - AWS CLI v2 (<https://aws.amazon.com/cli/>)
 - Git
 - Profile AWS `mktai` configurado (conta 124355648474)
-

4.1 Setup no macOS / Linux

Instalação:

```
cd mkt-digital-platform
npm install
npx prisma generate
npx prisma db push
npx tsx prisma/seed.ts
```

Executar:

```
AWS_PROFILE=mktai npm run dev -- --port 3030
```

4.2 Setup no Windows

Pré-requisitos Windows:

1. Instalar Node.js 20+ via <https://nodejs.org> (LTS)
2. Instalar Git via <https://git-scm.com/download/win>
3. Instalar AWS CLI v2 via <https://awscli.amazonaws.com/AWSCLIV2.msi>
4. Usar **PowerShell** ou **Terminal do Windows** (não usar cmd.exe antigo)

Configurar AWS CLI no Windows:

```
# Configurar o profile mktai
aws configure --profile mktai
# AWS Access Key ID: (inserir)
# AWS Secret Access Key: (inserir)
# Default region name: us-east-1
# Default output format: json

# Ou se usar ada/Isengard:
ada credentials update --profile=mktai --account=124355648474 --role=Admin --once

# Verificar:
aws sts get-caller-identity --profile mktai
```

Instalação no Windows:

```
cd mkt-digital-platform
npm install
npx prisma generate
npx prisma db push
npx tsx prisma/seed.ts
```

Executar no Windows (PowerShell):

```
$env:AWS_PROFILE="mktai"
npm run dev -- --port 3030
```

Executar no Windows (CMD):

```
set AWS_PROFILE=mktai
npm run dev -- --port 3030
```

Observacoes importantes para Windows:

- O script "dev": "AWS_PROFILE=mktai next dev" no package.json usa sintaxe Unix. No Windows, instale o pacote `cross-env` para compatibilidade:

```
npm install -D cross-env
```

E altere o script no package.json para:

```
"dev": "cross-env AWS_PROFILE=mktai next dev"
```

- Caminhos de arquivo usam `\` no Windows, mas o Node.js e Prisma lidam com isso automaticamente
- O SQLite funciona normalmente no Windows sem nenhuma configuração extra
- Se tiver problemas com `npx tsx`, use `npx ts-node --esm` como alternativa

4.3 Configuração do .env (ambos os sistemas):

Crie/edite o arquivo `.env` na raiz do projeto:

```
DATABASE_URL="file:./prisma/dev.db"
NEXTAUTH_SECRET="uma-chave-secreta-qualquer"
NEXTAUTH_URL="http://localhost:3030"
GOOGLE_CLIENT_ID="" # Opcional para dev
GOOGLE_CLIENT_SECRET="" # Opcional para dev
CRON_SECRET="uma-chave-para-cron"
AWS_PROFILE="mktai"
AWS_BEDROCK_REGION="us-east-1"
```

Executar:

```
AWS_PROFILE=mktai npm run dev -- --port 3030
```

Credenciais demo:

- Email: `demo@mktdigital.com`
- Senha: `demo123`

5. AWS Bedrock - Configuração

Profile AWS:

O projeto usa o profile `mktai` que aponta para a conta 124355648474.

Renovar credenciais:

```
ada credentials update --profile=mktai --account=124355648474 --role=Admin --once
```

Verificar:

```
aws sts get-caller-identity --profile mktai
```

Modelos necessários:

- `us.anthropic.claude-sonnet-4-6` (inference profile, us-east-1)
- `stability.stable-image-core-v1:1` (us-west-2)

Custos estimados:

Operação	Custo
Gerar 3 textos	~\$0.015 (varia com tokens)
Gerar 3 imagens	\$0.12 (\$0.04/imagem)
Total por post completo	~\$0.14

6. Banco de Dados

Schema principal (prisma/schema.prisma):

Tabela	Descrição
User	Usuários da plataforma
Account	Contas OAuth (Google)
Session	Sessões (JWT)
Company	Empresa do cliente (1:1 com User)
SocialAccount	Contas de redes sociais conectadas
Post	Posts criados (rascunho, agendado, publicado)
PostVariant	Variantes geradas (3 textos, 3 imagens)
CostLog	Registro de custos por geração

Migrar para PostgreSQL (produção):

- Alterar `provider = "sqlite"` para `provider = "postgresql"` no schema
- Alterar `colors String?` de volta para `colors Json?`
- Atualizar `DATABASE_URL` no `.env`
- Executar `npx prisma db push`

7. APIs - Referência

Autenticação

Rota	Método	Descrição
------	--------	-----------

/api/auth/[...nextauth]	GET/POST	NextAuth handlers
/api/auth/register	POST	Criar conta

Empresa

Rota	Método	Descrição
/api/company	GET	Dados da empresa
/api/company	POST	Criar/atualizar empresa
/api/company/logo	POST	Upload de logo

Geração de Conteúdo

Rota	Método	Body	Descrição
/api/generate/text	POST	{platform, idea?, topic?, trendingContext?, referenceImages?}	Gera 3 opções de texto
/api/generate/image	POST	{platform, idea?, style?, referenceContext?, trendingContext?}	Gera 3 imagens

Posts

Rota	Método	Descrição
/api/posts	GET	Listar posts
/api/posts	POST	Criar post
/api/posts/schedule	POST	Agendar post

Redes Sociais

Rota	Método	Descrição
/api/social/connect	POST	Conectar conta
/api/social/connect	DELETE	Desconectar conta
/api/social/publish	POST	Publicar post

Outros

Rota	Método	Descrição
/api/trends	GET	Buscar trending topics
/api/upload	POST	Upload de imagens de referência
/api/costs	GET	Consultar custos (params: period=week
/api/cron/publish	GET	Publicar posts agendados (requer Bearer token)

8. Fluxo de Publicação Automática

O endpoint `/api/cron/publish` deve ser chamado a cada 5 minutos por um serviço externo:

```
curl -H "Authorization: Bearer $CRON_SECRET" https://seu-dominio.com/api/cron/publish
```

Opções para produção:

- Vercel Cron Jobs
- AWS EventBridge + Lambda
- cron-job.org (grátis)

9. Próximos Passos (Roadmap)

Fase 4 - Geração de Vídeo

- Integrar HeyGen API para vídeos com avatar do cliente
- Integrar Runway Gen-3 para vídeos criativos
- Upload de vídeo base pelo cliente
- 3 opções de vídeo por geração

Fase 5 - Tráfego Pago

- Integrar Meta Ads API e Google Ads API
- Dashboard de métricas (impressões, cliques, conversões)
- Otimização automática de campanhas com IA
- A/B testing automático de criativos

Melhorias técnicas:

- Migrar de SQLite para PostgreSQL (Supabase ou Neon)
- Adicionar upload de imagens para S3 (em vez de filesystem)
- Implementar OAuth real para Instagram/Facebook/LinkedIn
- Adicionar testes automatizados (Jest + Playwright)
- Configurar CI/CD (GitHub Actions)
- Deploy em Vercel (front) + Railway (workers)
- Implementar sistema de planos/assinatura (Stripe)
- Rate limiting nas APIs
- Compressão de imagens geradas antes de salvar

10. Variáveis de Ambiente - Completo

```
# Banco de dados
DATABASE_URL="postgresql://user:pass@host:5432/mkt_digital"

# NextAuth
NEXTAUTH_SECRET="chave-segura-de-producao"
NEXTAUTH_URL="https://seu-dominio.com"

# Google OAuth
GOOGLE_CLIENT_ID="xxx.apps.googleusercontent.com"
GOOGLE_CLIENT_SECRET="xxx"

# AWS Bedrock
AWS_PROFILE="mktai"
AWS_BEDROCK_TEXT_REGION="us-east-1"
AWS_BEDROCK_IMAGE_REGION="us-west-2"
```

```
# Cron
CRON_SECRET="chave-segura-para-cron"
```

11. Comandos Úteis

macOS / Linux:

```
# Desenvolvimento
AWS_PROFILE=mktai npm run dev -- --port 3030

# Reset do banco (limpa tudo e recria com seed)
npm run db:reset

# Gerar client Prisma após alterar schema
npx prisma generate

# Push schema para banco
npx prisma db push

# Ver banco no browser
npx prisma studio

# Build de produção
npm run build

# Renovar credenciais AWS
ada credentials update --profile=mktai --account=124355648474 --role=Admin --once
```

Windows (PowerShell):

```
# Desenvolvimento
$env:AWS_PROFILE="mktai"
npm run dev -- --port 3030

# Reset do banco (Windows não tem rm, usar Remove-Item)
Remove-Item prisma\dev.db -ErrorAction SilentlyContinue
npx prisma db push
npx tsx prisma/seed.ts

# Gerar client Prisma após alterar schema
npx prisma generate

# Push schema para banco
npx prisma db push

# Ver banco no browser
npx prisma studio

# Build de produção
npm run build

# Renovar credenciais AWS
ada credentials update --profile=mktai --account=124355648474 --role=Admin --once
```


Windows (CMD):

```
:: Desenvolvimento
set AWS_PROFILE=mktai
npm run dev -- --port 3030

:: Reset do banco
del prisma\dev.db
npx prisma db push
npx tsx prisma/seed.ts
```

12. Troubleshooting

Erro de autenticação AWS

O processo Next.js herda `AWS_PROFILE` do shell. Sempre rode com:

```
AWS_PROFILE=mktai npm run dev
```

Erro "model not found" no Bedrock

- Texto usa inference profile `us.anthropic.claude-sonnet-4-6` (us-east-1)
- Imagem usa `stability.stable-image-core-v1:1` (us-west-2)
- Verifique se os modelos estão ativos na conta com `aws bedrock list-foundation-models`

Banco corrompido

```
# macOS/Linux:
rm prisma/dev.db && npm run setup

# Windows PowerShell:
Remove-Item prisma\dev.db; npm run setup
```

Imagens de referência não aparecem

Verifique se a pasta `public/uploads/` existe e tem permissão de escrita.

Windows: erro "AWS_PROFILE=mktai is not recognized"

O script dev no package.json usa sintaxe Unix. Solução:

```
npm install -D cross-env
```

E altere no package.json:

```
"dev": "cross-env AWS_PROFILE=mktai next dev"
```

Windows: erro "EACCES" ou permissão negada

Execute o terminal como Administrador, ou mude a pasta do projeto para fora de `C:\Program Files`.

Windows: erro "prisma db push" falha com SQLite

Certifique-se que nenhum outro processo (Prisma Studio, outro terminal) está usando o arquivo `dev.db`. Feche todos e tente novamente.

Fim do Manual